

ServiceDesk AI

A cloud-native customer support SaaS platform for small businesses

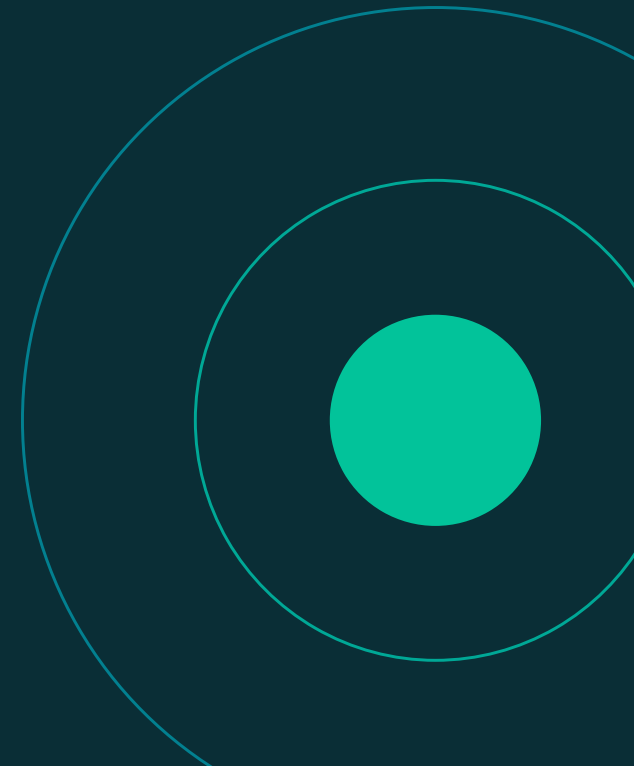
Project 19 • Customer Support SaaS • Review 1 • 20 August 2026

Aarush Jagannathan

23BLC1211

Gomatheswar M

23BLC1195



Problem Statement

“Manage customer complaints and automate ticket handling.”

Official problem statement — BCSE408L Project List, Project 19

What actually happens in a small business today



Requests arrive everywhere

Email, WhatsApp and phone calls.
There is no single place where a complaint is recorded.



Nobody owns the request

Without an assigned owner,
complaints are answered twice or
not at all.



No status, no history

The customer has no way to check
progress, so they call again —
creating more work.



No measurement

The business cannot answer how
many issues came in, or how long
they took to resolve.

Result: complaints are lost, response times are unknown, and the business has no evidence of how it is performing.

Existing System

Shared email inbox

The common approach

- No ticket identity or reference number
- No ownership — replies collide
- No status beyond read / unread
- History scattered across threads
- No reporting of any kind

Spreadsheet tracker

The manual workaround

- Updated by hand, so it goes stale
- No customer-facing view at all
- No access control — everyone edits everything
- Breaks down past a few dozen rows
- Not accessible outside the office

Enterprise helpdesk

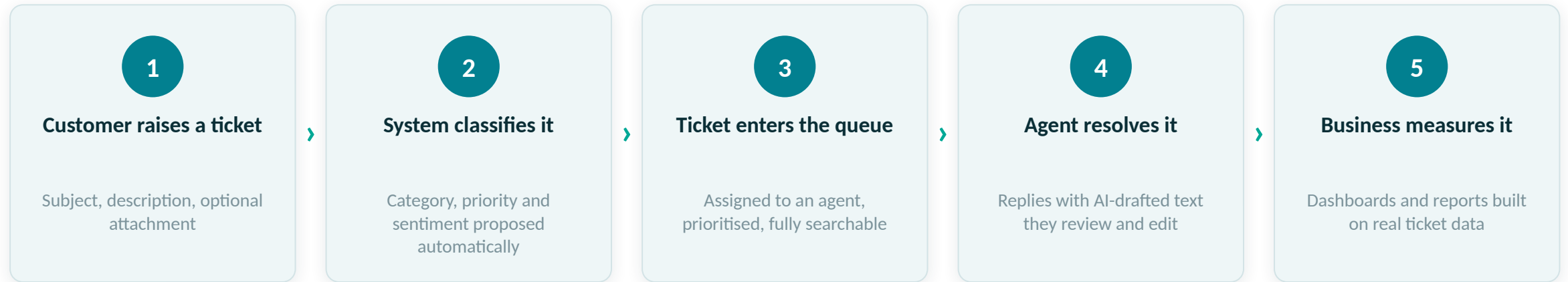
Zendesk, Freshdesk, Jira SM

- Priced per agent, per month
- Assumes a dedicated support team
- Long configuration before any value
- Far more features than a small team needs
- Cost scales faster than the business

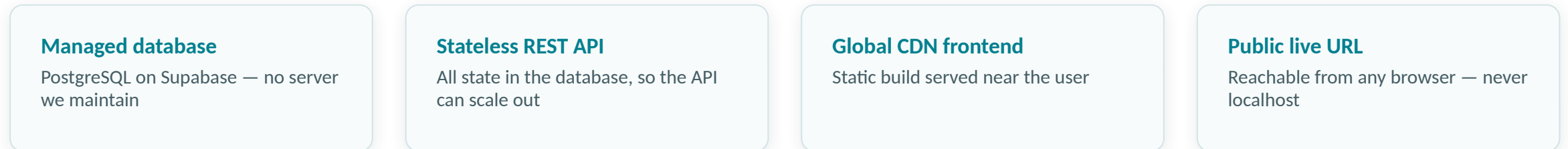
The gap: a small business needs helpdesk structure without helpdesk pricing or helpdesk complexity.

Proposed System

ServiceDesk AI is a multi-user, cloud-hosted SaaS application. Every complaint becomes a ticket with an owner, a category, a priority and a tracked status — visible to the customer, actionable by the agent, and measurable by the admin.



What makes it cloud-native



Objectives

The five objectives below are taken verbatim from the official Project 19 specification.



Develop a scalable cloud-native SaaS application

Three independently deployable tiers: React SPA, Express REST API, managed PostgreSQL.



Implement secure role-based authentication

JWT sessions with bcrypt-hashed passwords; roles enforced in the API, not the UI.



Provide dashboards and analytical reports

Role-specific dashboards and exportable reports computed from live ticket data.



Deploy the application on a public cloud platform

Vercel, Render and Supabase — a public URL, with no dependency on localhost.



Demonstrate collaborative software engineering practices

One team repository, regular commits, documented design, and tests.

Scope



In scope

Delivered by the final demonstration on 15 October

- Registration, login and profile management for three roles
- Role-based access control enforced in the REST API
- Full ticket lifecycle: create, read, update, delete, assign, transition
- Threaded customer-agent conversation with internal agent notes
- Search, filtering and pagination across all ticket fields
- Role-specific dashboards and exportable analytical reports
- Persistent in-app notifications
- AI classification, summarisation and agent-reviewed draft replies
- Admin modules: users, agents, categories, audit log
- Responsive interface for desktop, tablet and mobile
- Public cloud deployment with a live URL



Out of scope

Deliberate exclusions

- Payment or subscription billing
- Live chat and telephony integration
- Native mobile applications
- Multi-language support
- Third-party CRM integrations
- Automated SLA escalation engine
- Fully autonomous AI replies sent without agent approval

Excluded to keep the delivered system complete and reliable rather than broad and unfinished.

Target Users and Access Control



Customer

Raises and tracks complaints

- Register, log in, manage profile
- Create a support ticket
- View and search own tickets only
- Reply on a ticket thread
- Receive notifications

Cannot see any other customer's ticket, or any admin function.



Support Agent

Resolves tickets

- View the assigned ticket queue
- Search and filter all tickets
- Update status, priority, category
- Assign and escalate tickets
- Use the AI assistance panel

Cannot manage users, roles or categories.



Admin

Runs the service desk

- Manage users, agents and roles
- Manage categories and configuration
- View the global dashboard
- Generate and export reports
- Review the activity audit log

Full access, with every privileged action written to the audit log.

Access control is enforced in the API. Hiding a button in the interface is not security — every protected route checks the role, and every ticket handler checks record ownership.

Functional Requirements

The ten requirements below are the official Project 19 functional requirements. Each is mapped to the module that implements it.

FR-1	User registration and login Module: Authentication	FR-2	Profile management Module: User Management
FR-3	Role-based access control Module: Authentication	FR-4	CRUD operations Module: Core Business Module
FR-5	Search and filtering Module: Core Business Module	FR-6	Dashboard Module: Reports & Analytics
FR-7	Notifications Module: Core Business Module	FR-8	Report generation Module: Reports & Analytics
FR-9	Cloud database integration Module: Data Layer	FR-10	Administrator module Module: Administration

Module names match the Suggested Modules named in the project specification.

Non-Functional Requirements

The six quality attributes required by the specification, and how each is addressed in our design.



Security

Bcrypt password hashing, JWT sessions, role and ownership checks on every protected route, schema validation on every request, secrets in environment variables only.



Scalability

Stateless API holding no session state, so the platform can run multiple replicas; managed database connection pooling; static frontend on a CDN.



Availability

Managed cloud services with platform-level uptime; the application degrades gracefully when the AI provider is unreachable rather than failing.



Performance

Indexed queries on every filter column, full-text search index, server-side pagination, and cached client queries.



Reliability

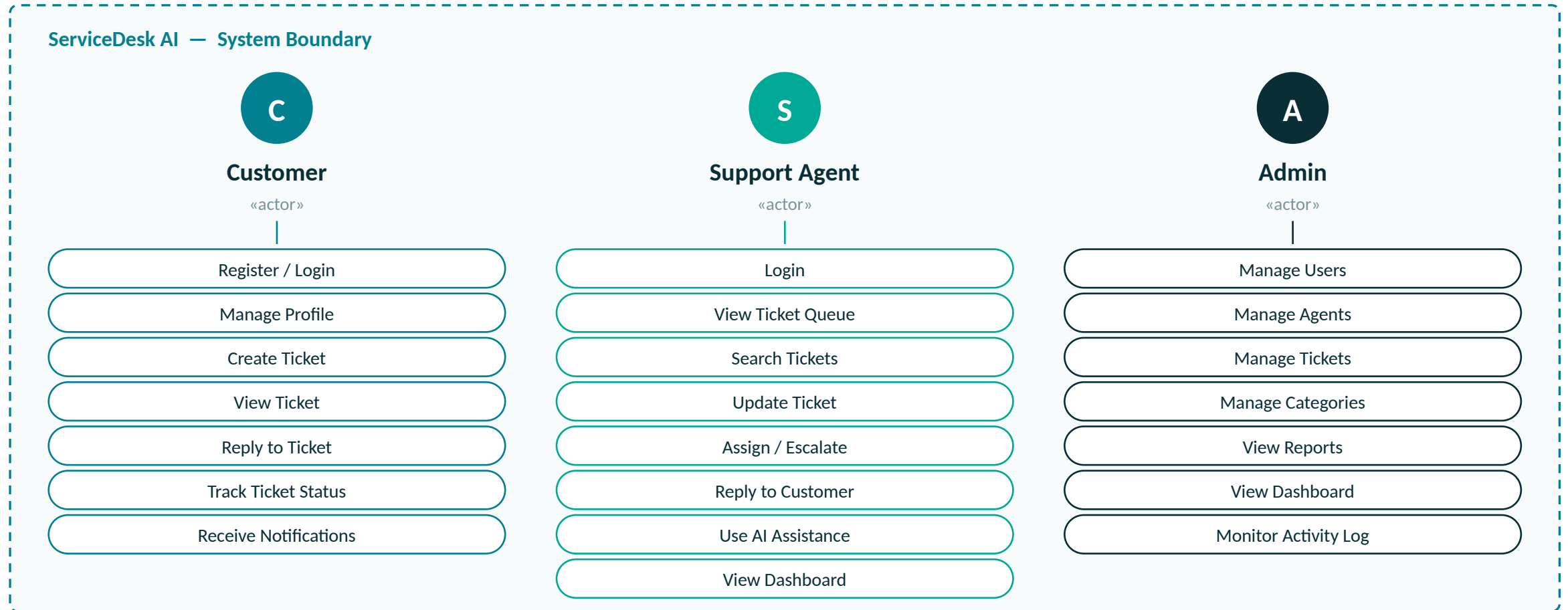
Database-level constraints, server-side lifecycle validation, structured error handling, and an audit log of privileged actions.



Usability

One consistent design system, responsive from 375 px upward, clear empty and error states, and no raw error text shown to users.

Use Case Diagram



Grouped by actor for readability. The full UML use case diagram with «include» and «extend» relationships is in Section 11 of the SRS.

Ticket Lifecycle

A ticket moves through six states. Valid transitions are defined once on the server and validated on every status change — an invalid transition is rejected, not silently applied.



Reopening

A RESOLVED ticket returns to OPEN if the customer replies that the issue persists.

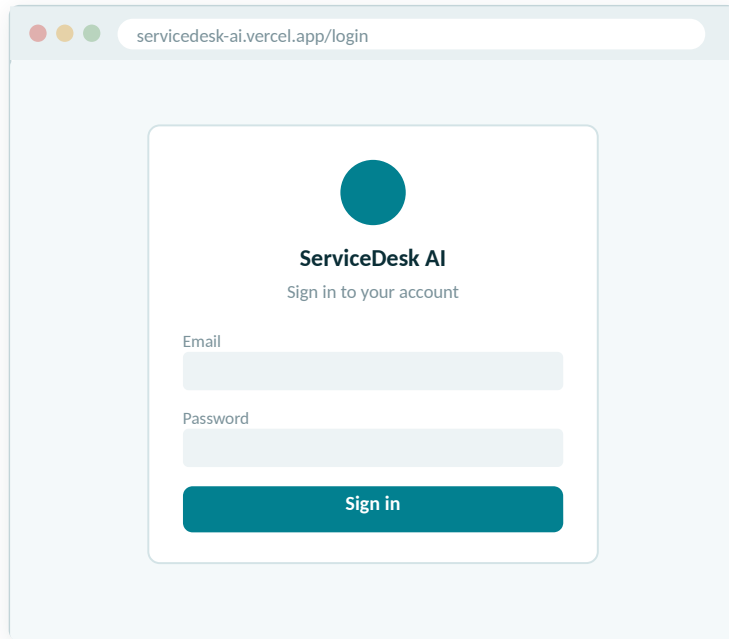
Timestamps

created_at, first_response_at and resolved_at are recorded to compute resolution time.

Audit trail

Every transition writes a ticket_events row: who changed what, from which value, when.

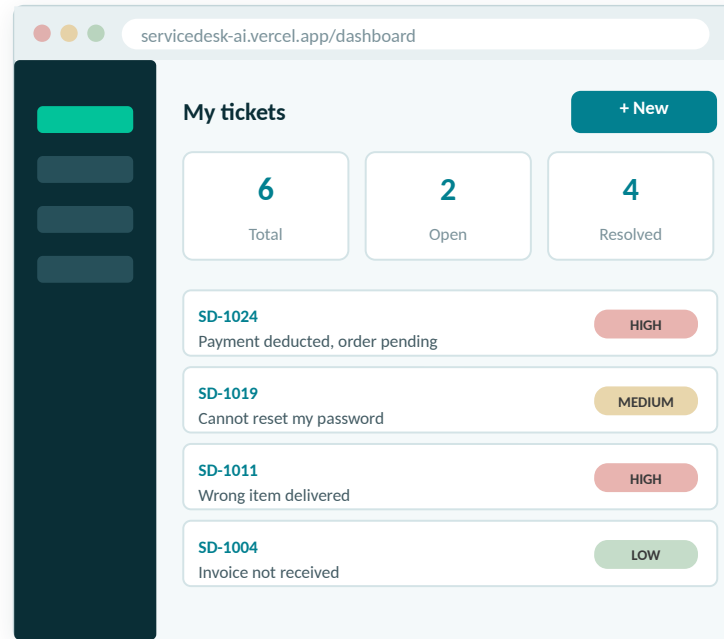
UI Mockups — Customer



The login form is centered on a light blue background. It features a teal circular logo at the top, followed by the text "ServiceDesk AI" and "Sign in to your account". Below this are two input fields: "Email" and "Password", each with a light gray border and a small teal validation icon on the right. A teal "Sign in" button is positioned at the bottom of the form.

Login

Email and password, with inline validation and clear error states. Registration follows the same layout.

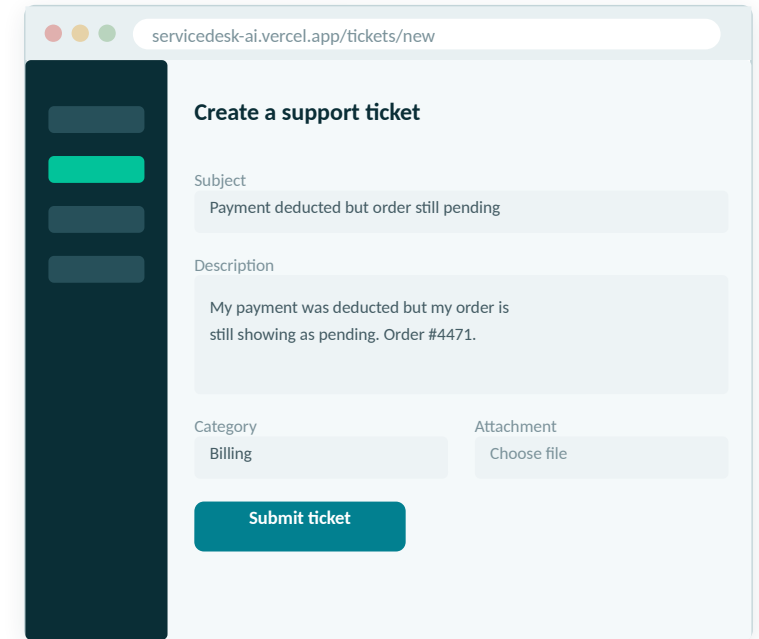


The dashboard has a dark teal sidebar with a teal "New" button. The main content area is titled "My tickets" and features a summary section with three cards: "Total" (6), "Open" (2), and "Resolved" (4). Below this is a list of tickets, each with a reference number, subject, and priority status.

Reference	Subject	Priority
SD-1024	Payment deducted, order pending	HIGH
SD-1019	Cannot reset my password	MEDIUM
SD-1011	Wrong item delivered	HIGH
SD-1004	Invoice not received	LOW

Customer Dashboard

Ticket counts computed from live data, then the customer's own tickets with reference, subject and priority.

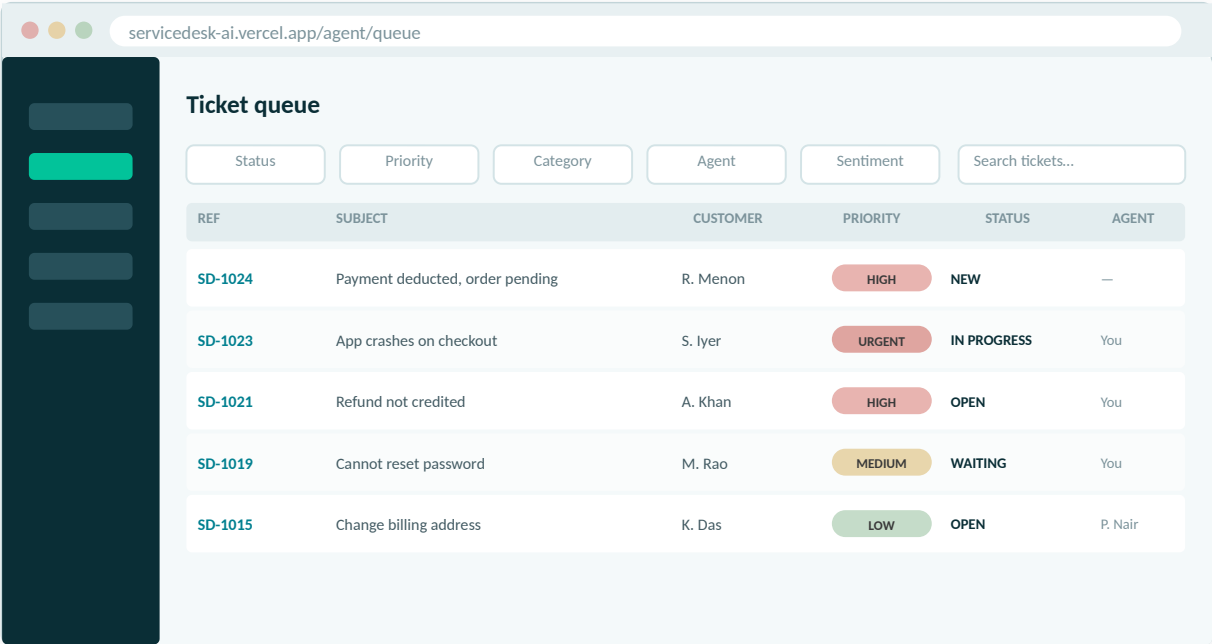


The form is titled "Create a support ticket". It includes a "Subject" field with the text "Payment deducted but order still pending", a "Description" field with the text "My payment was deducted but my order is still showing as pending. Order #4471.", a "Category" dropdown menu set to "Billing", and an "Attachment" section with a "Choose file" button. A teal "Submit ticket" button is at the bottom.

Create Ticket

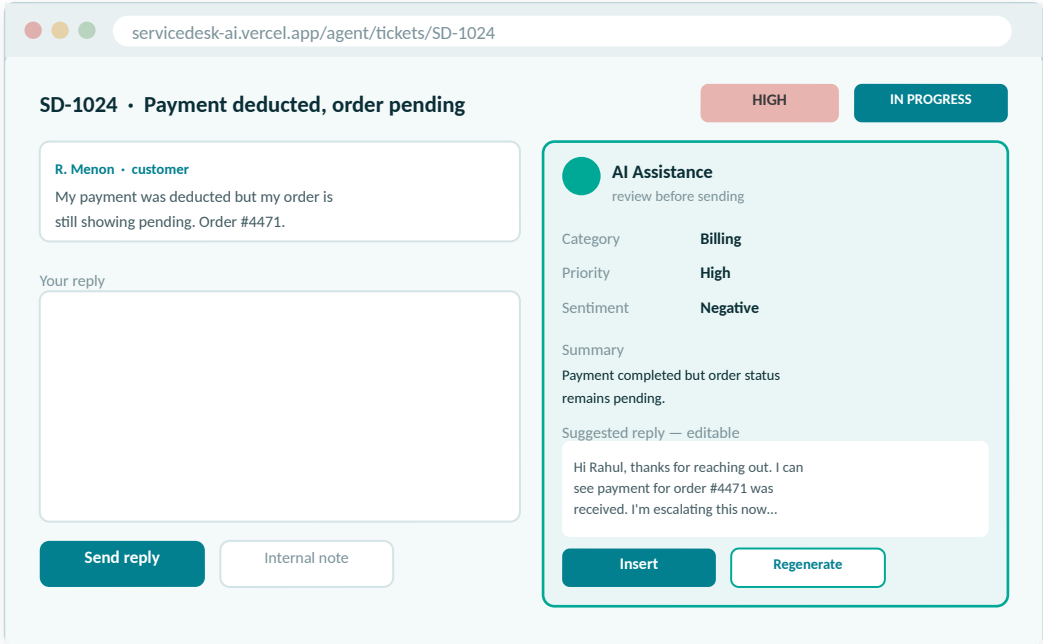
A short form. Category is optional — if left blank the AI proposes one, and the customer is never blocked waiting for it.

UI Mockups — Support Agent



Agent Ticket Queue

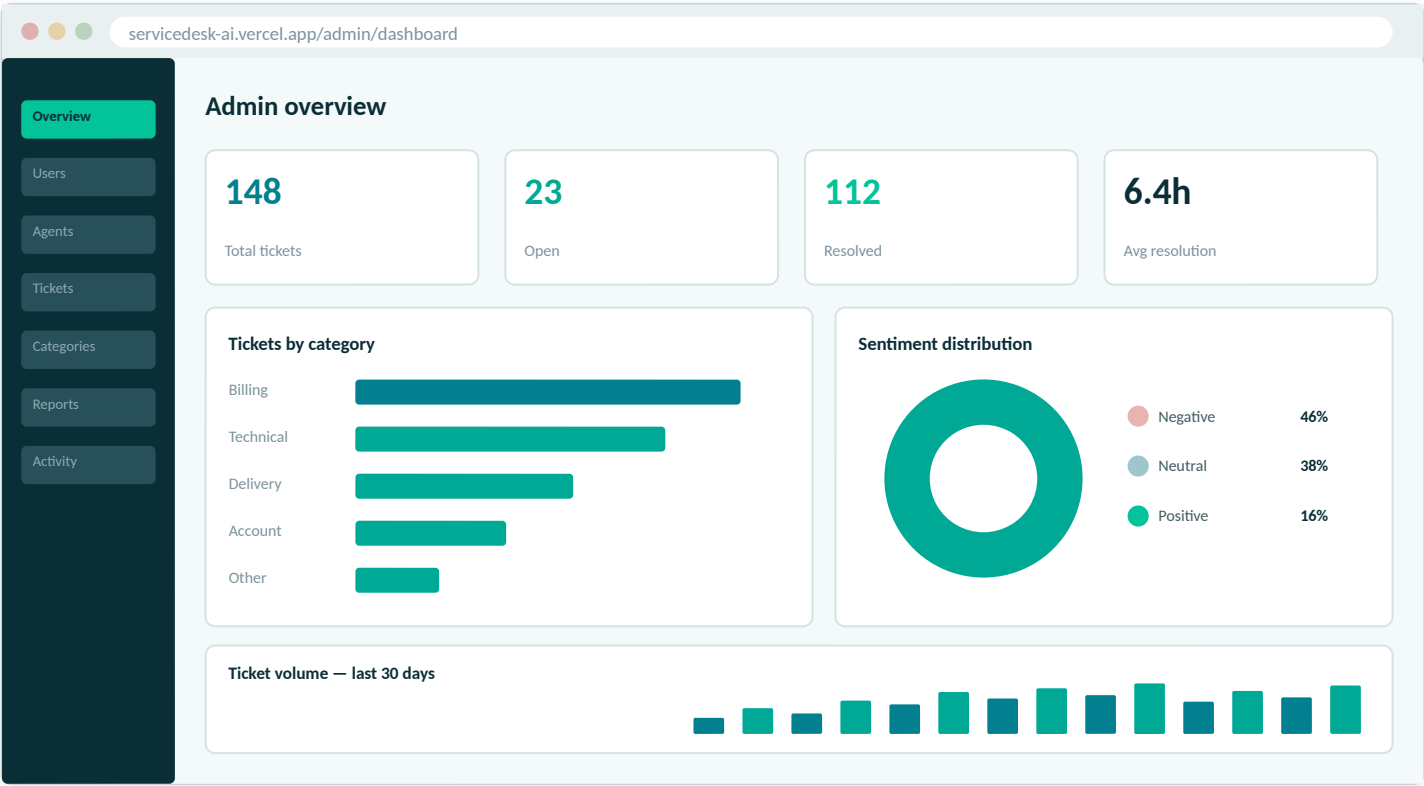
One prioritised list instead of an inbox. Filters on status, priority, category, agent and sentiment, plus full-text search — all applied server-side with pagination.



Ticket Detail with AI Assistance Panel

The AI proposes classification, a summary and a draft reply. Nothing is sent automatically — the agent inserts the draft into their own reply box, edits it, and sends.

UI Mockups — Admin



Admin Dashboard

Every number and every chart is computed from the database at request time. No dashboard figure is hard-coded.

Reports & export

Tickets by status, priority and category; agent workload; sentiment; volume trend; average resolution time — all exportable as CSV.

User & agent management

Create users, change roles, deactivate accounts. A deactivated user cannot log in but their ticket history is preserved.

Activity audit log

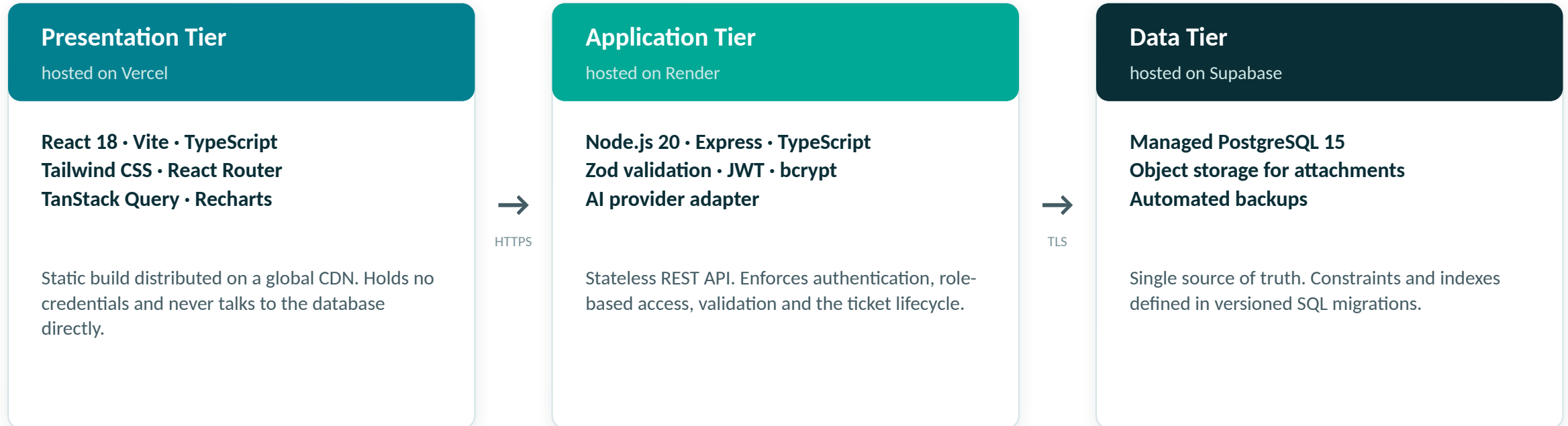
Every privileged action records actor, action, entity and timestamp — answering who changed this ticket, and when.

Responsive layout

The sidebar collapses to a bottom navigation bar below 768 px; tables become stacked cards on mobile.

Proposed Cloud Architecture

Three independently deployed tiers. Detailed system, cloud and database design is the Review 2 deliverable — this is the direction we have settled on.



Seven planned entities

users • categories • tickets • ticket_messages • ticket_events • notifications • audit_logs

The ER diagram will be generated from the migration SQL, so the diagram and the live database cannot drift apart.

AI-Assisted Ticket Handling

Honest scoping note: the official Project 19 specification does not require AI — only the project title contains it. Every AI capability here is our own addition, so no mandatory requirement is allowed to depend on it.

Classification

On submission, proposes category, priority and sentiment.

Human control: Agent can override every field.

Summarisation

Condenses a long conversation into one line.

Human control: Advisory only — never sent to a customer.

Draft reply

Generates a response for the agent to work from.

Human control: Agent must insert, edit and press send.

Resolution steps

Suggests troubleshooting steps for the issue.

Human control: Advisory only.

Worked example

Customer: "My payment was deducted but my order is still showing pending."

Category

Billing

Priority

High

Sentiment

Negative

Summary: Payment completed but order status remains pending.

What happens when the AI is unavailable

- Ticket creation still succeeds — AI runs after the ticket is saved.
- Falls back to a deterministic rule-based classifier.
- The panel shows a notice; the agent writes the reply.
- CRUD, dashboards and reports are unaffected.

Review Roadmap and Deliverables

R1

20 August

Requirements & Design

- Problem statement, existing and proposed system
- Functional and non-functional requirements
- Scope and use case diagram
- UI mockups and initial SRS
- Repository initialised

In progress

R2

17 September

Build & Deploy

- System, cloud and database architecture
- ER diagram and module design
- Authentication and role-based access
- Ticket CRUD and REST APIs
- Initial public cloud deployment

Planned

R3

15 October

Complete & Demonstrate

- Dashboards, reports and notifications
- AI assistance and error handling
- Responsive polish and testing
- Final report and presentation
- Live URL and working demonstration

Planned

Each of us can explain and modify any part of the system — module ownership rotates, and design decisions are documented as they are made.

Thank you

Questions are welcome — both of us are prepared to answer.

Project 19

ServiceDesk AI · Customer Support SaaS

Team

Aarush Jagannathan 23BLC1211 · Gomatheswar M 23BLC1195

Repository

github.com/ItzGOLLY/servicedesk-ai

Live URL

To be published at Review 2 — not yet deployed